

RBE549 P2: Building Built in Minutes - SfM and NeRF (Phase 2)

Nikesh Walling
Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA, USA
ngwalling@wpi.edu

Nehal Sonawane
Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA, USA
nvsonawane@wpi.edu

Neehal Sharrma
Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA, USA
nasharrma@wpi.edu

Abstract—This homework assignment explores Structure from Motion (SfM) and NeRF, techniques to reconstruct a 3D scene from a set of images and simultaneously obtain the camera poses of a monocular camera with respect to the given scene. This report adds to the existing content and covers phase 2 of the project, covering our NeRF implementation, the methodology, analysis and results.

Index Terms—Structure from Motion, NeRF, 3D Reconstruction

I. INTRODUCTION

3D reconstruction is the process of capturing the shape and appearance of real objects and scenes. By using multiple 2D images taken from different angles, algorithms can generate a 3D model that represents the structure, texture, and details of the original scene or object. This technique is widely used in computer vision, robotics, and augmented reality applications, enabling a more comprehensive understanding and interaction with the physical world.

II. STRUCTURE FROM MOTION

Structure from Motion (SfM) is a computer vision technique that aims to reconstruct the 3D structure of a scene from a set of 2D images. The basic idea is to estimate the camera poses and 3D points that best explain the observed 2D points in the images. This is a challenging problem, as it requires solving for the camera poses and 3D points simultaneously, and is sensitive to noise and outliers. To implement SfM the following steps are followed:

- Feature Matching and Outlier Rejection using RANSAC
- Estimating the Fundamental Matrix
- Estimating the Essential Matrix from the Fundamental Matrix
- Estimating Camera Poses from the Essential Matrix
- Checking the chirality Condition using Triangulation
- Perspective-n-Point
- Bundle Adjustment

A. Fundamental Matrix Estimation

The fundamental matrix describes the epipolar geometry between two images, relating the points in one image to the corresponding epipolar lines in the other image. This allows

us This relationship is captured in the following mathematical expression, referred to as the epipolar constraint:

$$\mathbf{x}'^T \mathbf{F} \mathbf{x} = 0 \quad (1)$$

where $\mathbf{x}$ and $\mathbf{x}'$ are the homogeneous coordinates of the corresponding points in the two images, and $\mathbf{F}$ is the fundamental matrix. Expanding further:

$$[x'_1, y'_1, 1] \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = 0 \quad (2)$$

which can be transformed into the following linear equation:

$$\mathbf{A} \mathbf{f} = 0 \quad (3)$$

where $\mathbf{A}$ is a matrix formed by stacking the corresponding points and $\mathbf{f}$ is a vector of the elements of $\mathbf{F}$.

To solve Equation (3), we use singular value decomposition (SVD) to find the least squares solution to the linear equation. The fundamental matrix is then constructed from the least squares solution, and the rank-2 constraint is enforced by setting the smallest singular value to zero.

To increase the stability of the solution, the points from each image are normalized using basic normalization matrices, $\mathbf{T}$ and $\mathbf{T}'$. Once the eight-point algorithm is applied to the normalized points, the fundamental matrix is denormalized using the following equation:

$$\mathbf{F}' = \mathbf{T}'^T \mathbf{F} \mathbf{T} \quad (4)$$

The eight-point algorithm is sensitive to noise and requires a sufficient number of point correspondences for accurate results. It can also be affected by degenerate configurations, such as coplanar points or degenerate camera motion. To address these issues, additional steps are required.

The resulting fundamental matrix can now be visualized through epipolar lines that pass through a feature point in one image and the location of the other camera in that image, depicted in Fig. 1



Fig. 1. Visualization of epipolar lines

B. RANSAC Outlier Optimization

We address the challenges of the naive eight-point algorithm by utilizing RANSAC to reject outlier correspondences, thereby obtaining a more accurate estimate of the fundamental matrix. RANSAC iteratively selects a random subset of the data and fits a model to that subset, in our case, a random set of 8 correspondences. We then evaluate the model on the remaining data and consider the points consistent with the model as inliers. We repeat this process for a specified number of iterations and select the model with the most inliers as the best estimate, as described in Algorithm 1:

Algorithm 1 RANSAC Estimation of Fundamental Matrix

```

1:  $n \leftarrow 0$ 
2: for  $i = 1$  to  $M$  do
3:   Choose 8 correspondences,  $\hat{x}_1$  and  $\hat{x}_2$ , randomly
4:    $F \leftarrow \text{EstimateFundamentalMatrix}(\hat{x}_1, \hat{x}_2)$ 
5:    $S \leftarrow \emptyset$ 
6:   for  $j = 1$  to  $N$  do
7:     if  $|\mathbf{x}_1^\top F \mathbf{x}_2| < \epsilon$  then
8:        $S \leftarrow S \cup \{j\}$ 
9:     end if
10:  end for
11:  if  $n < |S|$  then
12:     $n \leftarrow |S|$ 
13:     $S_n \leftarrow S$ 
14:  end if
15: end for

```

Evaluating the model on the remaining data is a crucial aspect of the algorithm. Instead of taking the naive approach of considering the algebraic error in (1) and applying a threshold, which does not accurately reflect the physical reality we are modeling, we focus on the geometric error. Therefore, we

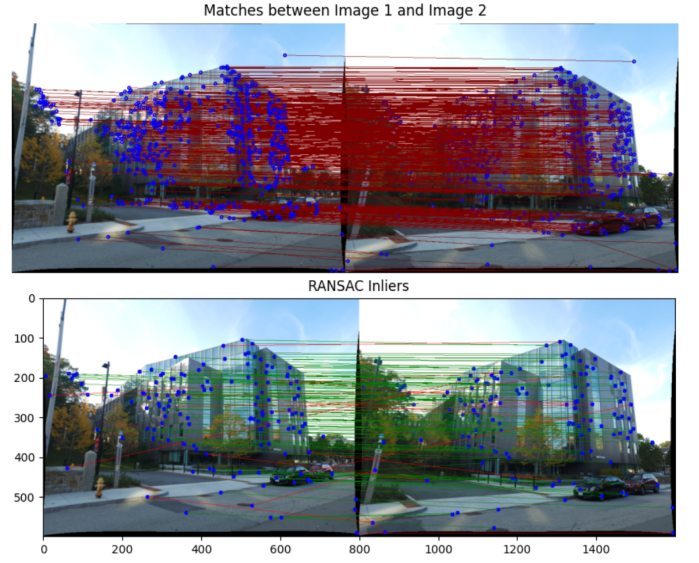


Fig. 2. Matched RANSAC features

use the Sampson distance, a first-order approximation to the geometric error, given by the following equation:

$$d(X_i, x_i)^2 = \frac{(x_i^\top \mathbf{F} X_i)^2}{(\mathbf{F} X_i)_1^2 + (\mathbf{F} X_i)_2^2 + (\mathbf{F}^\top x_i)_1^2 + (\mathbf{F}^\top x_i)_2^2} \quad (5)$$

The output of the fundamental matrix RANSAC with Sampson distance is shown in Figure 2. It is evident that RANSAC effectively removes the outlier correspondences, resulting in a more accurate set of matches.

C. Essential Matrix Estimation with Fundamental Matrix

The next step in the SfM pipeline involves estimating the essential matrix, a 3×3 matrix that relates the corresponding points in two images, assuming the cameras follow the pinhole model. We can compute the essential matrix from the fundamental matrix using the following relationship:

$$\mathbf{E} = \mathbf{K}^\top \mathbf{F} \mathbf{K} \quad (6)$$

Due to noise in calculating the fundamental matrix, the essential matrix may not be of rank 2. To enforce this constraint, we decompose the essential matrix using singular value decomposition and then reconstruct it using the following equation:

$$\mathbf{E} = \mathbf{U} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \mathbf{V}^\top \quad (7)$$

III. ESTIMATING CAMERA POSE FROM THE ESSENTIAL MATRIX

After estimating the essential matrix, we proceed to estimate the camera pose. The essential matrix represents the relative pose between two cameras, and we can decompose it into

the rotation and translation components. We use the following equation to perform this decomposition:

$$\mathbf{E} = [\mathbf{t}]_x \mathbf{R} \quad (8)$$

However, this decomposition is not unique and results in four possible camera poses. To resolve this ambiguity, we follow these steps:

- 1) Compute the four possible camera poses using the essential matrix.
- 2) Triangulate the 3D points for each of the four camera poses.
- 3) Enforce the chirality condition.

We begin by decomposing $\mathbf{E}$ using singular value decomposition (SVD) into $\mathbf{U}$, $\mathbf{D}$, and $\mathbf{V}$. We then compute the four possible camera poses using the following equations:

$$\mathbf{W} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (9)$$

$$\begin{aligned} \mathbf{R}_1 &= \mathbf{U}\mathbf{W}\mathbf{V}^\top & \mathbf{C}_1 &= \mathbf{U}(:, 3) \\ \mathbf{R}_2 &= \mathbf{U}\mathbf{W}\mathbf{V}^\top & \mathbf{C}_2 &= -\mathbf{U}(:, 3) \\ \mathbf{R}_3 &= \mathbf{U}\mathbf{W}^\top\mathbf{V}^\top & \mathbf{C}_3 &= \mathbf{U}(:, 3) \\ \mathbf{R}_4 &= \mathbf{U}\mathbf{W}^\top\mathbf{V}^\top & \mathbf{C}_4 &= -\mathbf{U}(:, 3) \end{aligned}$$

We then triangulate the 3D points for each possible pose (to be explained later).

To enforce the chirality condition in step 3, we ensure the 3D points are in front of the camera by checking the sign of the depth of the 3D points and discarding the camera poses that do not satisfy this condition. We use the following equation for the chirality condition:

$$\mathbf{R}(:, 3)^\top (\mathbf{X} - \mathbf{C}) > 0 \quad (10)$$

Due to noise, this equation may not be satisfied for all 3D points in the correct camera. To address this, we select the camera pose that satisfies the chirality condition for the maximum number of 3D points. We also adjust the threshold for the chirality condition to $\epsilon > 0.1$ to account for noise.

IV. LINEAR TRIANGULATION

With the estimated camera poses (R_i, C_i) and the corresponding 2D points in the images (x_i, x_j) , we proceed to calculate X , the world point for each correspondence. To solve the problem of triangulation, we start with the pinhole projection model:

$$x = PX \quad (11)$$

where P is the matrix:

$$P = K[R|t] \quad (12)$$

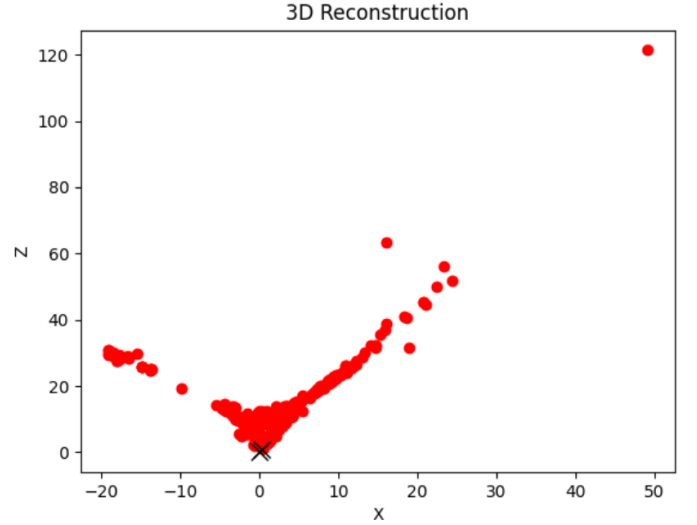


Fig. 3. Visualization of the triangulated points.

We can alternatively express this as:

$$x \times (PX) = 0 \quad (13)$$

We stack this equation for each camera pose and its corresponding image point, then solve for X using singular value decomposition. The result of this process is shown in Figure 4.

V. NON-LINEAR TRIANGULATION

With the linearly estimated 3D world points from the previous step, we refine their locations to minimize the reprojection error. The linear triangulation method minimizes the algebraic error, but the reprojection error is a more geometrically meaningful metric. This error is computed by measuring the geometric difference between an image point and the world point projected into its image plane.

Since camera models likely include nonlinearities, this method more accurately estimates the 3D points. Figure 4 shows a comparison of projections between nonlinear and linear triangulation methods.

This method improves the accuracy of the 3D points, as shown in the results section of this phase in Table I. However, the optimization step may not be necessary in all cases, as the initial error is often not significantly high. This reflects the fact that the linear triangulation method is already quite accurate.

VI. SfM PIPELINE FOR A SINGLE PAIR OF IMAGES

With all the building blocks in place, we can now summarize the SfM pipeline for a single pair of images:

- 1) Use RANSAC to reject outlier correspondences and find the fundamental matrix
- 2) Estimate the essential matrix from the fundamental matrix
- 3) Estimate the four possible camera poses from the essential matrix

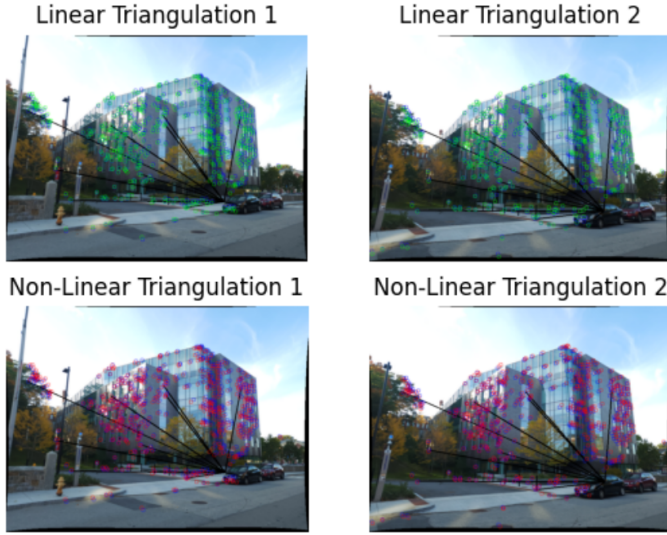


Fig. 4. Comparison of projections between non-linear and linear triangulation.

- 4) Linearly triangulate the 3D points for each possible pose
- 5) Disambiguate the camera poses using the cheirality condition
- 6) Non-linearly triangulate the 3D points to minimize the reprojection error

This pipeline works very well, and an output of the pipeline is shown in Figure 5.

VII. PERSPECTIVE-N-POINTS (PNP) AND PNP RANSAC

To extend this pipeline to multiple images, we need to estimate the camera pose for each image. We accomplish this using the Perspective-n-Points (PnP) algorithm, which estimates a camera pose from a set of 3D points and their corresponding 2D projections. The PnP algorithm can take in n points, hence the name, but we used $n = 6$ which allows the math to remain simple.

PnP first consists of forming a linear estimate of the camera pose by transforming Equation 10 into the following form:

$$\begin{bmatrix} P_{11} & P_{12} & P_{13} & P_{14} \\ P_{21} & P_{22} & P_{23} & P_{24} \\ P_{31} & P_{32} & P_{33} & P_{34} \end{bmatrix} \begin{bmatrix} X & Y & Z & 1 & 0 & 0 & 0 & 0 \\ -xZ & -x & & & & & & -xX \\ 0 & 0 & 0 & 0 & X & Y & Z & 1 \\ -yZ & -y & & & & & & -yX \end{bmatrix} \begin{bmatrix} -xX \\ -xY \\ -yX \\ -yY \end{bmatrix} \quad (14)$$

We then stack this equation n times for each point used in the linear PnP. Solving this using SVD yields the projection matrix P , which we decompose into R and t :

$$\begin{aligned} R &= K^{-1}P(:, 1:3) \\ t &= K^{-1}P(:, 4) \end{aligned} \quad (15)$$

We clean up the results using SVD:

$$\begin{aligned} [U, D, V] &= \text{SVD}(R) \\ R &= UV^T \\ t &= t/D_{11} \end{aligned} \quad (16)$$

Finally, if $\det(R) = -1$, we flip the sign of R and t .

This process provides a decent estimate of the camera pose but is sensitive to noise and outliers. To address this, we apply the RANSAC algorithm, selecting six random points each iteration and determining inliers using reprojection error. The result is shown in Figure 6.

VIII. NON-LINEAR PNP

While the linear PnP RANSAC algorithm is a good start, it is not perfect. To address this, we use the Levenberg-Marquardt algorithm to refine the camera pose estimate. The Levenberg-Marquardt algorithm is a non-linear optimization algorithm that minimizes the sum of squares of residuals. The residuals for the algorithm are given by the difference between the reprojected points and the true image points:

$$r = x - PX \quad (17)$$

This means that Levenberg-Marquardt minimizes the following cost function:

$$C = \sum_{i=1}^n \|x_i - PX_i\|^2 \quad (18)$$

The result of this process is shown in Figure 7.

IX. SfM PIPELINE FOR MULTIPLE IMAGES

With PnP added to our toolbox, we can now summarize the basic SfM pipeline for multiple images:

- 1) Perform the SfM pipeline for a single pair of images on the first two images
- 2) For each subsequent image:
 - a) Determine all image points which have a known 3D world-point
 - b) Estimate the camera pose using PnP RANSAC
 - c) Refine the camera pose using non-linear PnP
 - d) Triangulate all points which have a match to a previous image and do not already have a known 3D world-point
 - e) Add the new 3D points to the set of known world points

X. WORLD POINT SET DATA STRUCTURE

To efficiently determine which image points have a known 3D world-point, we store the world points in a data structure that keeps track of which image points are associated with each world point. This involves a pair of structures:

- 1) A list of world points and the associated image points in each image
- 2) A dictionary with the following structure:
 - a) Keys: image IDs (denoted as `id1`)

- b) Value: Dictionary
 - i) Keys: all image points ($ip1$) in image $id1$
 - ii) Value: Dictionary
 - A) Color: color of $ip1$
 - B) Has_WP: boolean indicating if $ip1$ has a known 3D world-point
 - C) Key: image ID ($id2$) of each image with a match to point $ip1$
 - D) Value: matched point ($ip2$) in image $id2$

These data structures allow us to efficiently recover all matches for a given point identified by an ($id1, ip1$) pair and to efficiently recover all matches for a given image identified by $id1$. The corresponding world point list tracks every world point and its color, as well as the image points in each image associated with it. This means the world point list encodes the tracks in the set of images.

A track is a set of image points all associated with the same world point. We use the world point list and the dictionary to efficiently determine which image points for a particular image have a known 3D world-point and which points matched within a point in a particular image do not have a known 3D world-point, crucial for the SfM pipeline for multiple images.

XI. BUNDLE ADJUSTMENT

The final aspect of the SfM pipeline is bundle adjustment, which refines the camera poses and 3D points simultaneously by minimizing the reprojection error of all the image points which have a known world point. We express the optimization problem as follows:

$$\min_{\{C_i, q_i, X_j\}} \sum_{i=1}^I \sum_{j=1}^J V_{ij} \|x_j - P_i X_j\|^2 \quad (19)$$

where C_i and q_i are the camera center and rotation quaternion (used for more stable convergence) for the i -th camera, X_j is the j -th world point, P_i is the projection matrix for the i -th camera, and V_{ij} is a binary matrix indicating if the j -th world point is visible in the i -th image.

This optimization problem is slow due to the large number of parameters but is made feasible by using a sparsity matrix for the Jacobian derived from V . We solve the optimization problem using the Trust Region Reflective algorithm.

We can now summarize our final SfM pipeline as follows:

- 1) Perform the SfM pipeline for a single pair of images on the first two images
- 2) For each subsequent image:
 - a) Determine all image points which have a known 3D world-point
 - b) Estimate the camera pose using PnP RANSAC
 - c) Refine the camera pose using non-linear PnP
 - d) Triangulate all points which have a match to a previous image and do not already have a known 3D world-point
 - e) Add the new 3D points to the set of known world points

- f) Perform bundle adjustment for all images/cameras and world points

XII. NEURAL RADIANCE FIELDS

In this phase, the primary objective is to implement NeRF. This involves the following steps:

- Data Loading
- Ray Generation
- Rendering
- Training and Testing

A. Data Loading

We begin by reading 2D images of a scene captured from different viewpoints; extracting camera poses (extrinsics) and intrinsic parameters (e.g., width, height, focal length); and finally preprocess the data into a format compatible with the pipeline. The Kaggle dataset has JSON values that we parse to extract camera angle, used to compute the focal length; and transformation matrices, to represent the camera's position and orientation in the world frame.

B. Ray Generation

Ray generation involves converting 2D pixel positions in an image into 3D rays in space. These rays are used to sample points in the 3D scene, which are then processed by the NeRF model to predict RGB colors and densities.

The ray generation process requires the following inputs:

- **Camera Intrinsics:** Image width (W), height (H), and focal length (focal)
- **Camera Extrinsics:** A 4x4 transformation matrix representing the camera's position and orientation in the world frame.
- **Pixel Positions:** The (x, y) coordinates of pixels in the 2D image.

C. Rendering

Rendering samples points along each ray using stratified sampling, queries the NeRF model to predict RGB values and density (sigma) for each sampled point, and performs volume rendering to compute the final RGB color for each ray. For each pixel in an image, a ray is generated in 3D space using the camera's intrinsic and extrinsic parameters. Points are sampled along each ray using stratified sampling. This involves dividing the ray into intervals and sampling points within each interval to ensure uniform coverage.

The predicted RGB values and densities are combined using *volume rendering* to compute the final color for the ray. Volume rendering integrates the contributions of all sampled points along the ray, taking into account their densities and distances. This process simulates how light interacts with the scene.

D. Training and Testing

The training process employed the Mean Squared Error (MSE) loss function to measure the difference between the predicted and ground truth RGB values. To improve efficiency, we used PyTorch’s GradScaler, which uses mixed precision sampling of operations to save memory usage and speed up computation by not precisely calculating every operation. The Adam optimizer is used to update the model parameters, with a learning rate of $5e-4$, and a learning rate scheduler for stable convergence.

During training, model checkpoints are saved every 100 iterations to allow resuming training if interrupted and to provide intermediate models for evaluation. For our implementation, we used a batch size of 4096 rays per batch ($32 \times 32 \times 4$), 400 samples per ray, and trained the model for 1,000 iterations total.

Training the NeRF model was expected to be the longest task. We weren’t able to tweak it enough to get great outputs even after a lot of training, but the results seem good enough that we’re sure that training it for longer will produce better results.

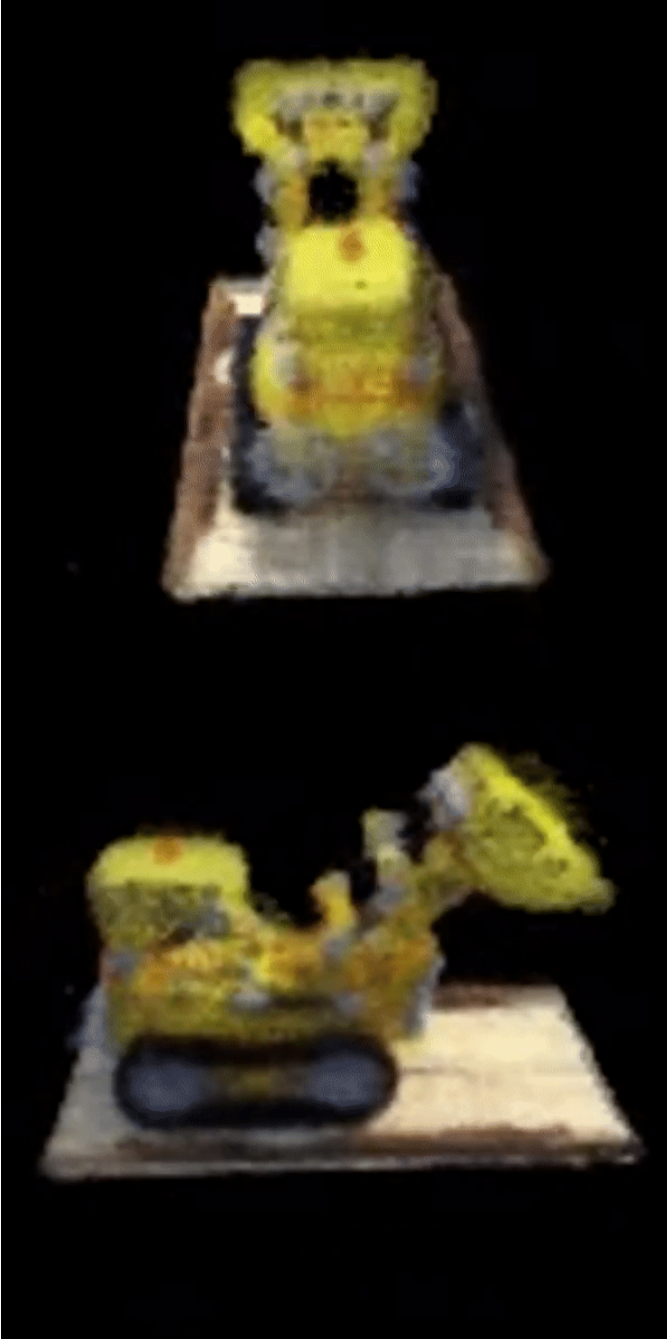


Fig. 5. Our NeRF output from the lego dataset.